Test Plan

> Covers the feedback system (FB-1 to FB-5) and the guided tour and version history (US-1 to US-8). Story identifiers follow the Requirements document and Requirements and Architecture: Guided Tour and Version History.

1. Approach

Testing is layered rather than uniform. Logic that can be exercised without a browser: the save format, arrow geometry and routing, cell-id parsing: is covered by unit tests, because those are the parts where a silent regression would be hardest to notice by hand. Behaviour that only makes sense as an interaction: arming a comment, linking a goal, entering staff mode: is covered by component tests driven through the rendered interface. Anything that depends on the model canvas rendering at a real size is verified by hand against fixtures, since maxGraph does not lay out under a headless renderer.

Whether the application can be used without instruction is checked with user experience testing (section 5), which no automated test can stand in for.

Three fixtures under `sample-models/` support the manual pass: a file in the pre-existing format with no feedback block, a file with comments and arrows, and a file with a grade as well.

The suite runs on every push through `.github/workflows/ci-cd.yml`, alongside a production build and the Cypress end-to-end suite. A failing test fails the pipeline. The guided tour does not open automatically under Cypress, so it never blocks an end-to-end run.

2. Coverage by user story

Counts are individual test cases, not files. “Manual” means the criterion is checked in the manual pass (section 4); “UX” means it is checked in user experience testing (section 5).

Feedback system

FB-1 Entering staff mode

Criterion	Where tested	Cases
Staff mode does not alter Create and Open	staffMode.test.tsx: offers only Create and Open until staff mode is on; adds Mark model once staff mode is on,	2

	leaving the others alone	
Confirmation then name, once per tab	feedbackFlow.test.tsx: teacher gate asks for confirmation then a name	1
Identity persists across files in one session	staffMode.test.tsx: keeps staff mode on when Open Model is used; shows who is signed in; turns staff mode off again from the same button	3

FB-2 Writing and managing feedback

Criterion	Where tested	Cases
A comment records its author and starts empty	feedbackTypes.test.ts: records the reviewer and starts with no comment and no arrows; gives each item a distinct id	2
Timestamp is written and readable	feedbackTypes.test.ts: formatFeedbackDate includes the year; falls back to the raw value when the timestamp cannot be parsed	2
Capped at twenty comments	feedbackColours.test.ts: caps comments at twenty	1
One consistent colour	feedbackColours.test.ts: uses the same blue for every comment	1

FB-3 Linking feedback to goals

Criterion	Where tested	Cases
Arming, linking, and staying armed for a second goal	feedbackFlow.test.tsx: links a goal, stays armed, and links a second goal; arming a box also selects it	2
Clicking a linked goal unlinks it	feedbackFlow.test.tsx: unlinks a goal when it is clicked again	1
Linking survives clicks that miss a goal	feedbackFlow.test.tsx: does not drop out of linking mode when a non-goal area is clicked; links on mousedown, because drag rows never fire a click	2
Goals are identified correctly on the canvas	graphAnchors.test.ts: reads the instanceId off a functional goal cell; handles the long timestamp ids the editor generates; reads every instanceId off a shared non-functional cell; ignores cells that are not goals; round trips ids produced by the editor; returns the goal a shape is anchored on; returns null for clicks that did not land on a goal	7
The click event name still matches the library	graphEventName.test.ts: GRAPH_CLICK_EVENT still matches maxGraph	1

FB-4 Reading feedback

Criterion	Where tested	Cases
Only the selected comment draws arrows	feedbackGeometry.test.ts: draws nothing when no comment is selected; draws only the selected comment; returns nothing when no comment is selected; returns the targets of the selected comment; returns nothing for a comment that no longer exists. feedbackArrows.test.tsx: draws nothing until a comment is selected; emits a path element for the selected comment	7
One arrow per linked goal, correctly placed	feedbackGeometry.test.ts: produces one line per link, relative to the container; draws one line per target when a comment points at several goals; offsets coordinates when the container is not at the page origin; carries the comment and goal ids on every line	4
A goal that is gone or off screen does not break the view	feedbackGeometry.test.ts: skips a link whose goal is not on screen; skips a box that is not rendered; returns	3

	nothing for comments with no links	
Arrows are distinguishable from the model's own edges	feedbackGeometry.test.ts: uses a dash long enough not to be mistaken for the model; stays thin enough that overlapping arrows remain readable; emits a cubic curve between the two endpoints; bends the curve when the two ends are vertically aligned; keeps both control points between the ends, whichever way round they are	5
Arrows avoid other shapes rather than crossing labels	arrowRouting.test.ts: the full routeArrow, segmentHitsRect and routeToPath suites, including that routing terminates on a crowded diagram rather than searching forever	15

FB-5 Saving and persistence

Criterion	Where tested	Cases
Files predating the capability open unchanged	feedbackTypes.test.ts: returns null for a file saved before the feedback feature existed; is false when there is no block at all	2

A malformed block is ignored, not fatal	feedbackTypes.test.ts: treats a malformed block as absent rather than throwing; rejects an item whose instanceId is not in n-n form	2
Reviewed-but-empty is distinguishable from never reviewed	feedbackTypes.test.ts: is false for a file explicitly stamped unfeedbacked; is true for a reviewed file even when the reviewer cleared every comment; stamps unfeedbacked when the model was never reviewed; stamps feedbacked when a reviewer was present	4
Data survives a save and reload	feedbackTypes.test.ts: reads back a well formed block; writes a timestamp that survives a round trip through the schema	2

Guided tour and version history

US-1 Guided tour opens for a new user (AMBR-21)

Criterion	Where tested	Cases
The tour opens automatically when the user enters the editor	Manual pass: guided tour	Manual
Auto-open is skipped with ?tour=off and under a Cypress test	Manual pass: guided tour; Cypress runs in CI	Manual

---

A Guide button in the editor header starts the tour at any time	Manual pass: guided tour	Manual
A new user can build a model without reading the manual	User experience testing: three participants, no prior instruction	UX

US-2 Each step highlights what it explains (AMBR-22)

Criterion	Where tested	Cases
Steps cover the five goal categories, adding a goal, arranging the hierarchy, the rendered model, view controls, and saving	Manual pass: guided tour	Manual
Steps shown match the current page (Goal List or Render Model)	Manual pass: guided tour	Manual
The highlighted element is the one the step explains	User experience testing	UX

US-3 The tour never blocks work (AMBR-22)

Criterion	Where tested	Cases
The tour can continue to the Render Model page and pick up there, with the graph centred	Manual pass: guided tour	Manual
The tour can be closed or skipped at any step	Manual pass: guided tour	Manual

US-4 Automatic checkpoint on save (AMBR-36)

Criterion	Where tested	Cases
Clicking Save creates an automatic checkpoint labelled with the date and time	Manual pass: version history	Manual

US-5 Named checkpoint (AMBR-37)

Criterion	Where tested	Cases
Saving with a name adds the version to the list, marked Named	Manual pass: version history	Manual
Named checkpoints are kept when the oldest automatic checkpoints are removed	Manual pass: version history	Manual

US-6 List of saved versions (AMBR-38)

Criterion	Where tested	Cases
Versions are listed newest first, marked Auto or Named	Manual pass: version history	Manual
History is saved inside the model file, so it stays with the model even if the file is renamed or moved	Manual pass: version history	Manual

US-7 Restore a previous version (AMBR-39)

Criterion	Where tested	Cases
Restoring a version from the list returns the model to that state	Manual pass: version history	Manual

---

The user is asked to confirm before the current model is replaced	Manual pass: version history	Manual

US-8 Rename or delete a saved version (AMBR-40)

Criterion	Where tested	Cases
A version can be renamed and deleted from the list	Manual pass: version history	Manual
Up to 50 versions are kept; when full, the oldest automatic ones are removed first	Manual pass: version history	Manual

Grading (not required: kept on the grade-system branch)

The client confirmed in Week 5 that grading is not needed. The branch and its tests are kept only as a record and are outside the project scope.

Criterion	Where tested	Cases
The grade view appears only once something is entered	grade.test.ts: is false for an untouched form, so no button appears; is false when there is no grade at all; is true once any part has been filled in	3
Scores are computed and stored correctly	grade.test.ts: rounds to one decimal place; returns null rather than NaN when no maximum was set; round trips through the schema; still parses a file with no grade	4

The 500-character limit holds at the boundary	grade.test.ts: rejects overall feedback beyond the limit; accepts overall feedback exactly at the limit	2
Score entry behaves under real typing	numberField.test.tsx: clearing without a zero jumping back in; multi-digit entry one key at a time; empty treated as zero on blur; commits on Enter so a value is not lost by pressing Save straight away; clamps below the minimum rather than storing a negative mark; ignores text that is not a number; follows the parent while not being edited	7

3. Automated test totals (feedback capability)

File	Cases
`feedbackGeometry.test.ts`	17
`arrowRouting.test.ts`	15
`feedbackTypes.test.ts`	14
`grade.test.ts`	9
`numberField.test.tsx`	7
`graphAnchors.test.ts`	7
`feedbackFlow.test.tsx`	6
`staffMode.test.tsx`	5

`feedbackColours.test.ts`	2
`feedbackArrows.test.tsx`	2
`graphEventName.test.ts`	1

Latest CI run (GitHub Actions): Vitest 15 files, 134 tests passing; Cypress 2 specifications passing.

Eleven test files for the feedback capability. At the end of Week 6, run together with the pre-existing suite, the project reported 15 files and 161 passing tests, against 4 files and 60 at the start of the project.

4. Manual verification

Anything that depends on the model canvas laying out at a real size cannot be driven by the automated suite, so it is verified by hand. Each capability is exercised through the interface, and the paths through it are combined against each other rather than tried one at a time.

Feedback system

- Create and Open, and the staff-mode entry, in each order and after one another in the same tab.
- Entering and leaving staff mode, opening several files in one session, and confirming the session ends on refresh.
- Creating, editing and deleting comments, then saving and reopening to confirm they return with their author and date.
- Linking and unlinking against goals on the rendered model and rows in the hierarchy, including arming a comment and clicking somewhere that is not a goal.
- Selecting each comment in turn to confirm only its arrows draw, as a student as well as a reviewer.
- Opening each of the three fixtures: the pre-existing format, a file with comments and arrows, and a file with a grade.
- Resizing the window, scrolling and zooming the canvas, and collapsing the sidebar, confirming arrows track the goals they point at.
- The sidebar and canvas reorder in GraphWorker.tsx, including resize detection and recentring after the change.

Guided tour

- Entering the editor with and without `?tour=off`, confirming the tour opens only when it should.
- Starting the tour from the Guide button on the Goal List and on the Render Model page.

- Stepping through every step, confirming each highlights the element it explains and fits on one line.
- Continuing from the Goal List steps to the Render Model page, confirming the graph is centred and the tour picks up there.
- Closing and skipping the tour at each step, confirming the editor is usable straight away.
- Leaving the editor mid-tour, confirming the tour is removed.

Version history

- Saving repeatedly, confirming each save adds an automatic checkpoint with the date and time.
- Saving a named checkpoint, then renaming and deleting versions from the list.
- Restoring an earlier version and confirming the model returns to that state.
- Renaming and moving the model file, then reopening it to confirm the history is still there.
- Opening a model file saved before version history existed, confirming it opens and is given a model ID.
- Saving past the 50-version limit, confirming the oldest automatic versions are removed first and named ones are kept.
- Opening the same file in another browser, confirming the history comes with it.

Interface changes

- Opening and closing the Papers modal.
- Each keyboard shortcut, including Ctrl+E then P or S for direct export, and confirming shortcuts do not fire while an input field has focus.
- Empty-state guidance on the canvas and the tree, and the highlight while dragging.
- Toast notifications after save, duplicate and export.

Combinations of the above are tried in whatever order occurs to the person testing, rather than as a fixed script. The pass is run before each client demonstration, including the final review of every feature with the client on 15 Oct, and after any change that touches the canvas. What it finds is fixed rather than logged for later.

5. User experience testing

Three participants who know, or can follow, motivational models used the updated AMMBER with no prior explanation of how to use it, and were then asked for their impressions. This checks what the automated and manual passes cannot: whether a first-time user can find their way through the editor, the guided tour and version history on their own.

Sessions, findings and the changes made in response: User experience testing

6. Limits of the current coverage

Recorded so a reader knows what the suite does and does not carry on its own.

- GraphWorker.tsx has no automated coverage. Its behaviour after the sidebar reorder is verified by hand every time the file changes, so a regression there would be caught by a person rather than by the pipeline.
- The guided tour and version history are verified by the manual pass and user experience testing, not by automated tests.
- The exploratory pass is not scripted. It is thorough, but it is not identical between runs and not reproducible by someone who did not carry it out, so it does not protect against regression the way a recorded case does.
- Cypress carries the pre-existing specifications only. No end-to-end scenario yet drives the feedback capability, the guided tour or version history through to a saved file.
- Arrow routing is a bounded heuristic with a cap of four detours. It has been exercised by hand on the fixtures, but the cap has not been measured against a model dense enough to reach it.

The next step is writing the exploratory pass up as Cypress scenarios, which would make the coverage repeatable rather than broader.